PYTHON PROGRAMMING
CASE STUDIES

Assignment Documentation

Student Name	_____
Roll Number	_____
Branch / Section	_____
Subject	Python Programming

This document contains the problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python code, test cases, sample output screenshots and conclusion for each case study.

Contents

1. Project 27 – Advanced Numeric Pattern Generator
2. Project 28 – Repeated Transaction Detection
3. Project 29 – Password Policy Validation
4. Project 30 – Student Email and Identifier Validation
5. Project 35 – Text Compression Using Character Runs
6. Project 36 – Repeated Word and Phrase Detection
7. Project 37 – Nested Loop Calendar Generator
8. Project 42 – Student Marks Matrix Analysis
9. Project 45 – Weekly Electricity Consumption Matrix
10. Project 49 – Tuple-Based Product Catalogue Audit

Project 27 – Advanced Numeric Pattern Generator

Concept: Nested loops and conditional statements

1. Problem Statement

Write a Python program to generate a diamond-shaped star pattern for a size entered by the user. The program must reject a size that is zero or negative.

2. Objectives

- Practice nested for loops.
- Use conditions to validate input.
- Print spaces and stars in the correct order.

3. Assumptions

- The size is an integer.
- The size must be greater than zero.

4. Algorithm

1. Start the program.

2. Read the diamond size n.

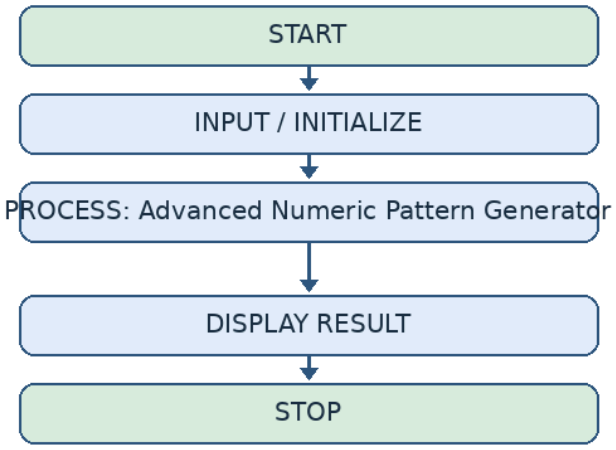
3. If n is less than or equal to zero, display an invalid-size message.

4. Print the upper half using increasing rows of stars.

5. Print the lower half using decreasing rows of stars.

6. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
IF n <= 0
    PRINT "Invalid size"
ELSE
    FOR i = 1 TO n
```

```
        PRINT (n-i) spaces and (2*i-1) stars
    FOR i = n-1 DOWNTO 1
        PRINT (n-i) spaces and (2*i-1) stars
STOP
```

7. Python Code

```
n = int(input("Enter diamond size: "))

if n <= 0:
    print("Invalid size")
else:
    for i in range(1, n + 1):
        for j in range(n - i):
            print(" ", end="")
        for j in range(2 * i - 1):
            print("*", end="")
        print()

    for i in range(n - 1, 0, -1):
        for j in range(n - i):
            print(" ", end="")
        for j in range(2 * i - 1):
            print("*", end="")
        print()
```

8. Test Cases

Test	Input	Expected Result
1	3	Diamond with 3 rows in the upper half
2	0	Invalid size
3	-2	Invalid size

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program uses nested loops to print a symmetric diamond pattern and validates the entered size.

Project 28 – Repeated Transaction Detection

Concept: Lists, loops and duplicate detection

1. Problem Statement

Create a transaction report using transaction IDs and amounts. Identify repeated transaction IDs and display transactions whose amount is greater than 1000.

2. Objectives

- Store transaction details in lists.
- Compare IDs using nested loops.
- Filter high-value transactions using a condition.

3. Assumptions

- The two lists contain the same number of items.
- A repeated ID is treated as a duplicate transaction.
- A high-value transaction has an amount greater than 1000.

4. Algorithm

1. Create lists for transaction IDs and amounts.

2. Display every transaction.

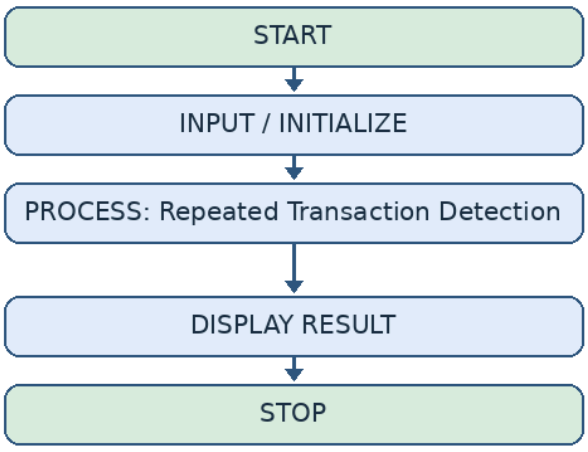
3. Compare each ID with the IDs that follow it.

4. Print an ID when a match is found.

5. Check every amount and print amounts greater than 1000.

6. Stop.

5. Flowchart



6. Pseudocode

```
START
SET ids and amounts
DISPLAY transaction report
FOR each pair of positions i, j where j > i
    IF ids[i] = ids[j], DISPLAY duplicate ID
```

```
FOR each amount
    IF amount > 1000, DISPLAY transaction
STOP
```

7. Python Code

```
ids = [101, 102, 103, 101, 104, 102]
amounts = [500, 1200, 800, 500, 2000, 1200]

print("Transaction Report")
for i in range(len(ids)):
    print(ids[i], amounts[i])

print("\nDuplicate Transactions:")
for i in range(len(ids)):
    for j in range(i + 1, len(ids)):
        if ids[i] == ids[j]:
            print("Duplicate ID:", ids[i])

print("\nHigh Value Transactions:")
for i in range(len(amounts)):
    if amounts[i] > 1000:
        print(ids[i], amounts[i])
```

8. Test Cases

Test	Input	Expected Result
1	IDs [101,102,101]; amounts [500,1200,500]	Duplicate ID 101; transaction 102 with amount 1200 is high value
2	No repeated IDs	No duplicate ID is displayed

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program detects repeated transaction IDs and separately reports all transactions above the given amount limit.

Project 29 – Password Policy Validation

Concept: Strings, loops and conditions

1. Problem Statement

Write a program that checks whether a password follows a basic policy: at least eight characters, one uppercase letter, one lowercase letter, one digit and one special character. Reject common patterns such as '123' and 'password'.

2. Objectives

- Examine each character in a string.
- Count uppercase, lowercase, digit and special characters.
- Apply multiple password validation conditions.

3. Assumptions

- The password is entered as text.
- A special character is any character that is not a letter or digit.
- The program checks simple rules and is not a complete security system.

4. Algorithm

1. Read the password.

2. Initialize four counters to zero.

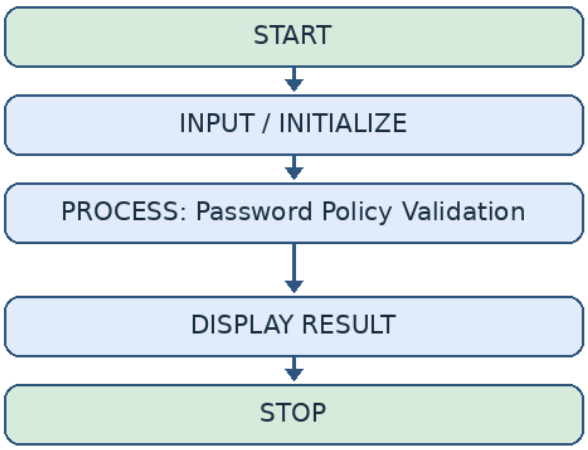
3. Visit each character and update the correct counter.

4. Display a message for every rule that is not satisfied.

5. Check for prohibited patterns.

6. If all rules pass, display that the password is valid.

5. Flowchart



6. Pseudocode

```
START
INPUT password
SET upper, lower, digit, special = 0
FOR each character ch
    COUNT its character type
```

```
CHECK length and all counters
CHECK prohibited patterns
IF every rule passes, PRINT valid
STOP
```

7. Python Code

```
password = input("Enter password: ")

upper = 0
lower = 0
digit = 0
special = 0

for ch in password:
    if ch.isupper():
        upper += 1
    elif ch.islower():
        lower += 1
    elif ch.isdigit():
        digit += 1
    else:
        special += 1

valid = True

if len(password) < 8:
    print("Password must contain 8 characters")
    valid = False
if upper == 0:
    print("Add one uppercase letter")
    valid = False
if lower == 0:
    print("Add one lowercase letter")
    valid = False
if digit == 0:
    print("Add one number")
    valid = False
if special == 0:
    print("Add one special character")
    valid = False
if "123" in password or "password" in password.lower():
    print("Contains a prohibited pattern")
    valid = False

if valid:
    print("Password is valid")
```

8. Test Cases

Test	Input	Expected Result
1	Abc@1234	Contains a prohibited pattern; invalid
2	Hello@45	Password is valid
3	hello	Messages for missing rules; invalid

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program checks password strength using character counters and conditional statements. It demonstrates basic validation only.

Project 30 – Student Email and Identifier Validation

Concept: String operations and conditions

1. Problem Statement

Accept a student's email address and student ID. Check whether the email has a simple valid structure and whether the ID follows the format CSE-26-101.

2. Objectives

- Use string operators and methods.
- Validate the position and count of symbols in an email.
- Split an identifier and validate its parts.

3. Assumptions

- The email must contain exactly one @ symbol and a dot after it.
- The student ID has three parts separated by hyphens.
- The ID format is letters, two digits and three digits.

4. Algorithm

1. Read the email and student ID.

2. Check the email conditions.

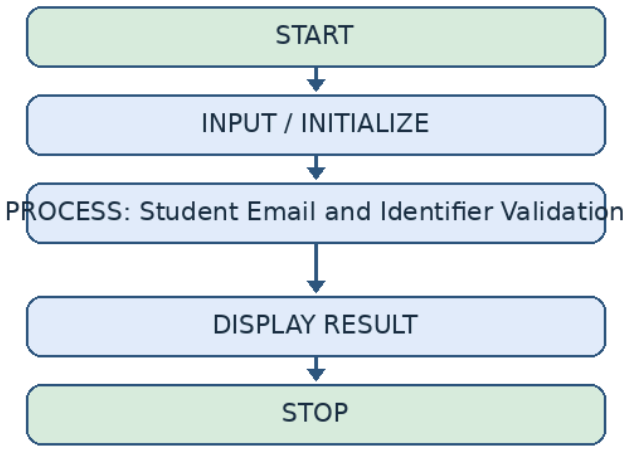
3. Display valid or invalid email.

4. Split the student ID using '-'.

5. Check the number and format of its parts.

6. Display valid or invalid student ID.

5. Flowchart



6. Pseudocode

```
START
INPUT email, student_id
CHECK @ count, dot position and spaces
DISPLAY email result
SPLIT student_id by '-'
```

```
CHECK 3 parts: letters, 2 digits, 3 digits
DISPLAY ID result
STOP
```

7. Python Code

```
email = input("Enter student email: ")
student_id = input("Enter student ID: ")

# Email validation
if ("@" in email and
        email.count("@") == 1 and
        "." in email and
        email.index(".") > email.index("@") + 1 and
        email[-1] != "." and
        " " not in email):
    print("Valid Email")
else:
    print("Invalid Email")

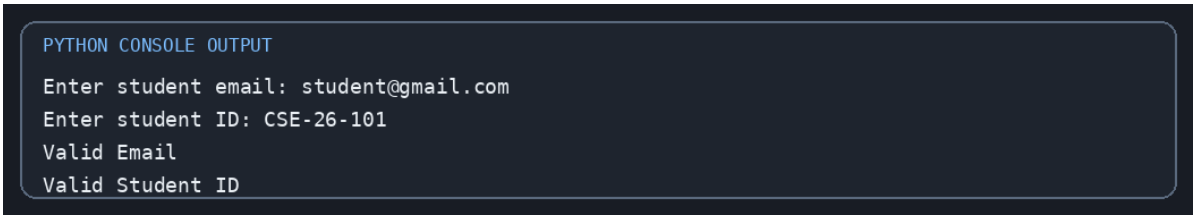
# Student ID format: CSE-26-101
parts = student_id.split("-")

if (len(parts) == 3 and
        len(parts[0]) >= 2 and
        len(parts[1]) == 2 and
        len(parts[2]) == 3 and
        parts[0].isalpha() and
        parts[1].isdigit() and
        parts[2].isdigit()):
    print("Valid Student ID")
else:
    print("Invalid Student ID")
```

8. Test Cases

Test	Input	Expected Result
1	student@gmail.com, CSE-26-101	Valid Email; Valid Student ID
2	studentgmail.com, CSE-26-1	Invalid Email; Invalid Student ID

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program applies simple string checks to validate student email addresses and identifiers.

Project 35 – Text Compression Using Character Runs

Concept: Strings, loops and pattern processing

1. Problem Statement

Compress a string by replacing each consecutive group of the same character with the character followed by its count. Compare the original and compressed lengths.

2. Objectives

- Traverse a string using a loop.
- Count consecutive repeated characters.
- Build a compressed string and compare lengths.

3. Assumptions

- The input is a string.
- Each run is represented as character followed by count, including count 1.
- This simple method may make some strings longer.

4. Algorithm

1. Read the input string.

2. Set result to an empty string and count to one.

3. Compare each character with the previous character.

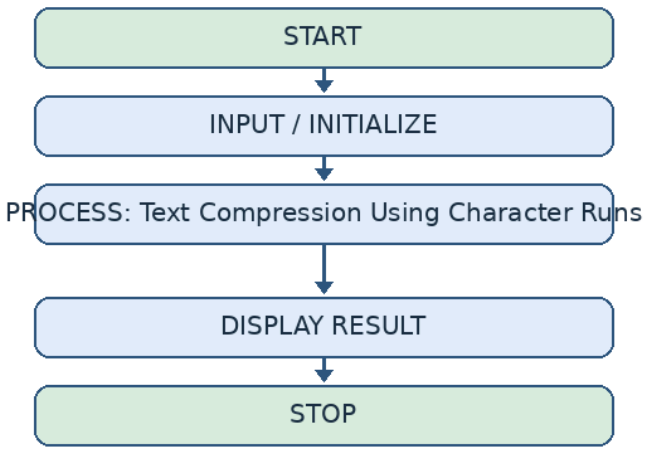
4. Increase count when characters match; otherwise append the previous character and count.

5. Append the final character run.

6. Display both strings and lengths.

7. Report whether compression is beneficial.

5. Flowchart



6. Pseudocode

```
START
INPUT text
SET result = empty, count = 1
```

```
FOR i from 1 to length(text)-1
    IF text[i] = text[i-1], count = count + 1
    ELSE append text[i-1] and count; reset count
IF text is not empty, append last character and count
DISPLAY original and compressed strings and lengths
COMPARE lengths
STOP
```

7. Python Code

```
text = input("Enter a string: ")

result = ""
count = 1

for i in range(1, len(text)):
    if text[i] == text[i - 1]:
        count += 1
    else:
        result = result + text[i - 1] + str(count)
        count = 1

if len(text) > 0:
    result = result + text[-1] + str(count)

print("Original Text:", text)
print("Compressed Text:", result)
print("Original Length:", len(text))
print("Compressed Length:", len(result))

if len(result) < len(text):
    print("Compression is beneficial")
else:
    print("Compression is not beneficial")
```

8. Test Cases

Test	Input	Expected Result
1	aaabbc	a3b2c1; compression is not beneficial
2	aaaaabbbb	a5b4; compression is beneficial
3	empty string	Empty compressed string; no crash

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program performs run-length style compression by counting adjacent repeated characters and compares the result size.

Project 36 – Repeated Word and Phrase Detection

Concept: Strings, lists and loops

1. Problem Statement

Read a paragraph, identify words that appear more than once, and display the total number of words.

2. Objectives

- Convert text to lowercase for comparison.
- Split a paragraph into words.
- Count repeated words without printing the same repeated word more than once.

3. Assumptions

- Words are separated by spaces.
- Punctuation is not removed in this basic version.
- Uppercase and lowercase versions of a word are treated as the same.

4. Algorithm

1. Read a paragraph.

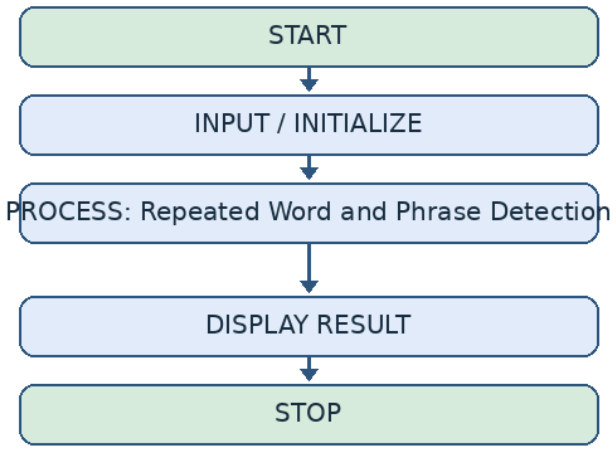
2. Convert it to lowercase and split it into a list of words.

3. For each word, count how many times it occurs.

4. Print the word only if it occurs more than once and has not appeared earlier in the list.

5. Display the total word count.

5. Flowchart



6. Pseudocode

```
START
INPUT sentence
words = lowercase(sentence).split()
FOR each word at position i
    COUNT matching words in the list
    IF count > 1 and word not in words before i
        DISPLAY word and count
```

```
DISPLAY number of words
STOP
```

7. Python Code

```
sentence = input("Enter a paragraph: ")
words = sentence.lower().split()

print("\nRepeated Words:")
for i in range(len(words)):
    count = 0
    for j in range(len(words)):
        if words[i] == words[j]:
            count += 1

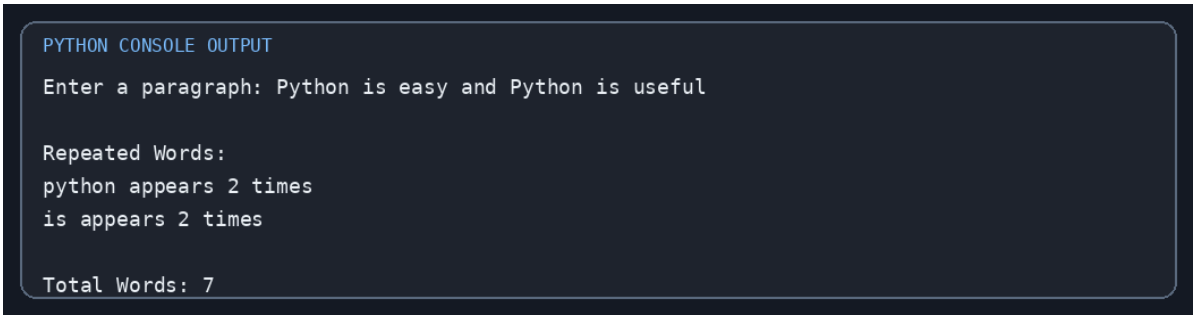
    if count > 1 and words[i] not in words[:i]:
        print(words[i], "appears", count, "times")

print("\nTotal Words:", len(words))
```

8. Test Cases

Test	Input	Expected Result
1	Python is easy and Python is useful	python appears 2 times; is appears 2 times; total 7
2	This is simple	No repeated words; total 3

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program uses string splitting and nested loops to find repeated words in a paragraph.

Project 37 – Nested Loop Calendar Generator

Concept: Nested loops, conditions and formatting

1. Problem Statement

Generate a simple monthly calendar for a month with 28 to 31 days. The user also enters the weekday on which the month starts.

2. Objectives

- Use loops to arrange calendar dates.
- Use a starting weekday offset.
- Format dates into weekday columns.

3. Assumptions

- The month has between 28 and 31 days.
- Monday is represented by 0 and Sunday by 6.
- The program does not calculate months or leap years automatically.

4. Algorithm

1. Read number of days and starting weekday.

2. Validate both values.

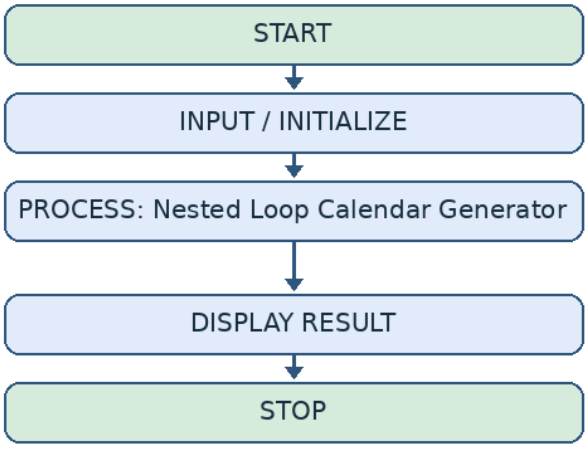
3. Print weekday headings.

4. Print blank spaces for days before the first date.

5. Print dates from 1 to the given number of days.

6. Move to a new line after every seven calendar positions.

5. Flowchart



6. Pseudocode

```
START
INPUT days, start
IF days outside 28..31 OR start outside 0..6
    PRINT Invalid input
ELSE
```

```
    PRINT weekday headings
    PRINT start blank positions
    FOR day = 1 TO days
        PRINT day in a 3-character column
        IF (start + day) is divisible by 7, PRINT newline
STOP
```

7. Python Code

```
days = int(input("Enter number of days: "))
start = int(input("Starting weekday (0=Mon, 6=Sun): "))

if days < 28 or days > 31 or start < 0 or start > 6:
    print("Invalid input")
else:
    print("MON TUE WED THU FRI SAT SUN")

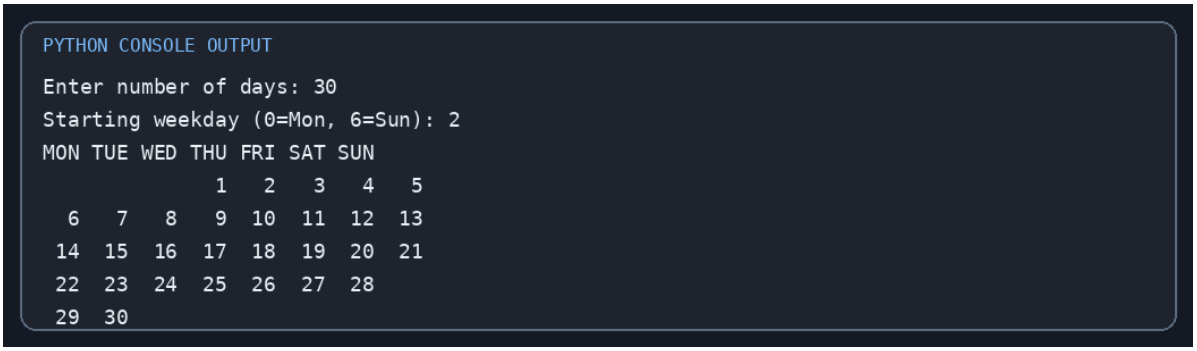
    for i in range(start):
        print("    ", end="")

    for day in range(1, days + 1):
        print("%3d " % day, end="")
        if (start + day) % 7 == 0:
            print()
    print()
```

8. Test Cases

Test	Input	Expected Result
1	30 days, start 2	Calendar begins under Wednesday and contains dates 1 to 30
2	25 days, start 0	Invalid input
3	31 days, start 7	Invalid input

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program generates a basic calendar layout using loops and a weekday offset.

Project 42 – Student Marks Matrix Analysis

Concept: Nested lists, matrices and row-wise/column-wise calculations

1. Problem Statement

Read marks for multiple students and subjects into a matrix. Calculate each student's average, each subject's average, and identify the student with the highest total marks.

2. Objectives

- Store marks in a two-dimensional list.
- Calculate row and column averages.
- Find the student with the highest total.

3. Assumptions

- The number of students and subjects must be positive integers.
- Marks are entered as numbers.
- All students have marks for the same number of subjects.

4. Algorithm

1. Read the number of students and subjects.

2. Create an empty matrix.

3. Read marks for each student and append each row.

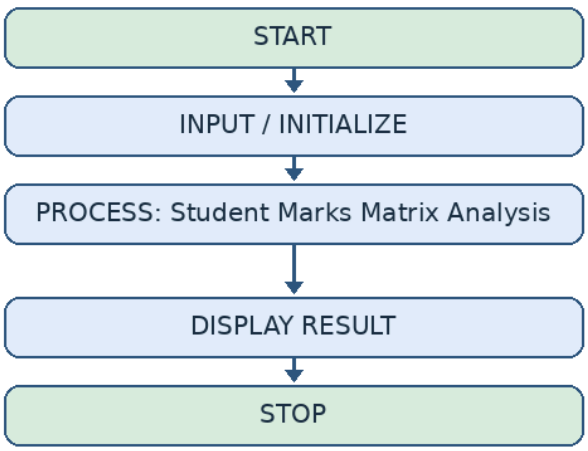
4. Display the marks matrix.

5. Calculate and display each student's average.

6. Calculate and display each subject's average.

7. Find and display the student with the highest total marks.

5. Flowchart



6. Pseudocode

```
START
INPUT students, subjects
CREATE empty marks matrix
```

```
FOR each student
    INPUT marks for every subject
    ADD row to matrix
DISPLAY matrix
CALCULATE each row average
CALCULATE each column average
FIND row with maximum total
DISPLAY topper and total
STOP
```

7. Python Code

```
students = int(input("Enter number of students: "))
subjects = int(input("Enter number of subjects: "))

marks = []

for i in range(students):
    row = []
    print("\nStudent", i + 1)
    for j in range(subjects):
        m = int(input("Enter marks: "))
        row.append(m)
    marks.append(row)

print("\nStudent Marks Matrix:")
for row in marks:
    print(row)

print("\nStudent Averages:")
for i in range(students):
    total = 0
    for j in range(subjects):
        total += marks[i][j]
    avg = total / subjects
    print("Student", i + 1, "Average:", avg)

print("\nSubject Averages:")
for j in range(subjects):
    total = 0
    for i in range(students):
        total += marks[i][j]
    avg = total / students
    print("Subject", j + 1, "Average:", avg)

top = -1
topper = 0
for i in range(students):
    total = 0
    for j in range(subjects):
        total += marks[i][j]
    if total > top:
        top = total
        topper = i + 1

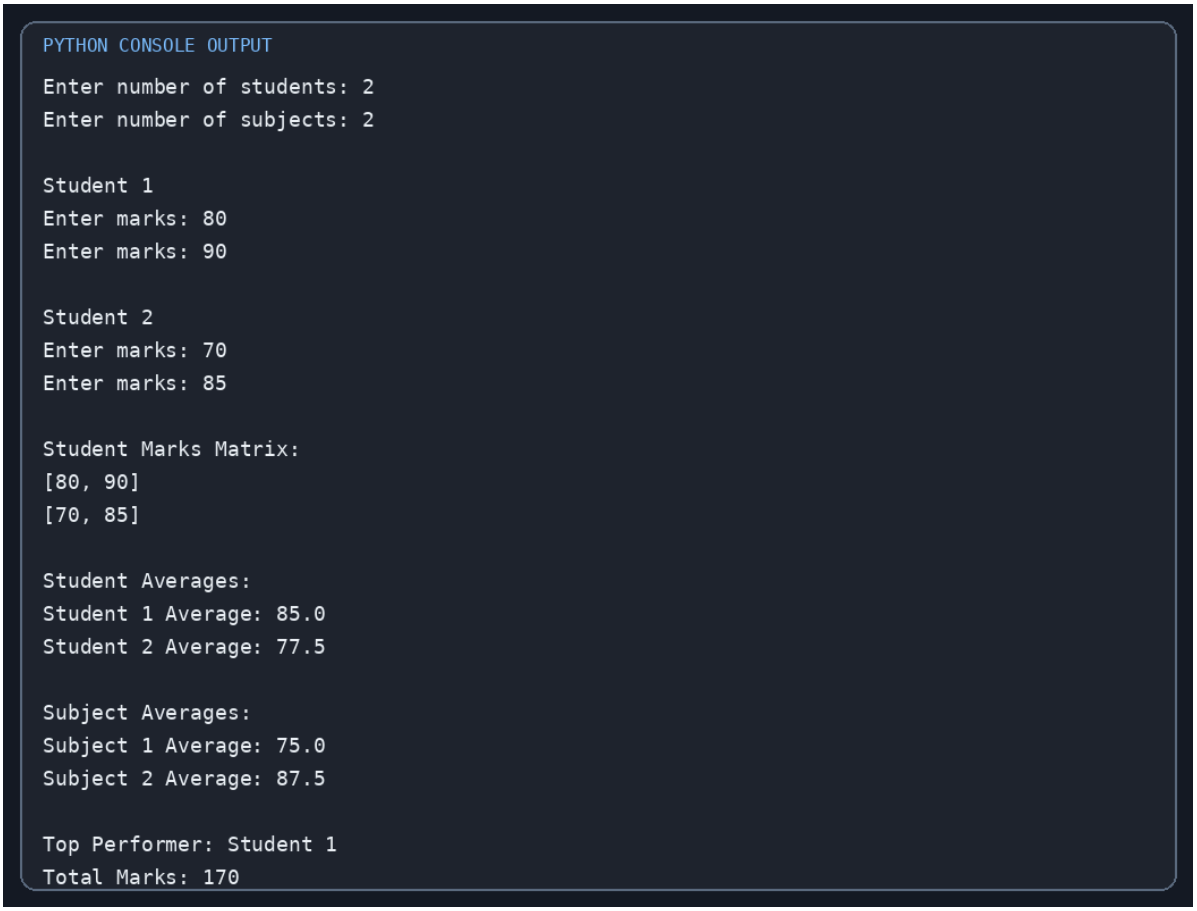
print("\nTop Performer: Student", topper)
print("Total Marks:", top)
```

8. Test Cases

Test	Input	Expected Result

1	2 students, 2 subjects: [80,90], [70,85]	Student averages 85 and 77.5; subject averages 75 and 87.5; Student 1 topper with 170
2	0 students	Input should be positive; not valid for meaningful averages

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program demonstrates two-dimensional lists and performs student-wise and subject-wise analysis.

Project 45 – Weekly Electricity Consumption Matrix

Concept: Nested lists, loops, totals and conditions

1. Problem Statement

Record the daily electricity usage of four appliances for seven days. Calculate the weekly total for each appliance and daily total consumption. Display a warning when a day's total is greater than 20 units.

2. Objectives

- Store weekly usage in a two-dimensional list.
- Calculate appliance-wise and day-wise totals.
- Use a condition to identify high consumption.

3. Assumptions

- There are four appliances: Fan, Light, TV and AC.
- Usage is entered in units and can contain decimals.
- A daily total above 20 units is considered high.

4. Algorithm

1. Create the appliance list and an empty usage matrix.

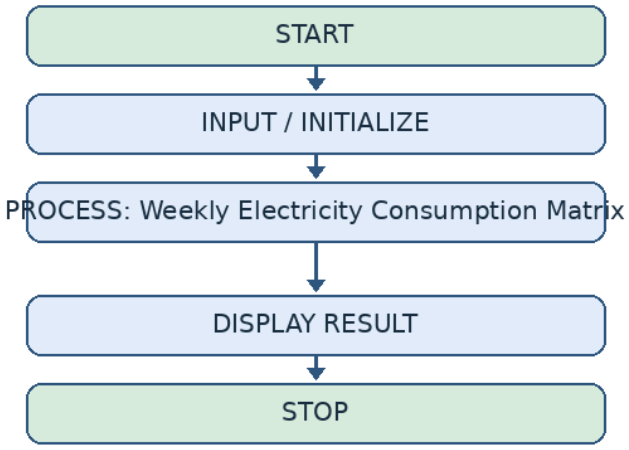
2. For each appliance, read usage for seven days.

3. Calculate and display the weekly total for each appliance.

4. For each day, add usage of all appliances.

5. Display the daily total and show a warning if it is above 20.

5. Flowchart



6. Pseudocode

```
START
SET appliances = Fan, Light, TV, AC
INPUT usage for 4 appliances over 7 days
FOR each appliance
    CALCULATE sum of its 7 values
    DISPLAY weekly total
FOR each day
```

```
    CALCULATE sum across all appliances
    DISPLAY daily total
    IF total > 20, DISPLAY high consumption warning
STOP
```

7. Python Code

```
appliances = ["Fan", "Light", "TV", "AC"]
usage = []

for i in range(4):
    row = []
    print("\nEnter", appliances[i], "usage for 7 days")
    for j in range(7):
        units = float(input("Enter units: "))
        row.append(units)
    usage.append(row)

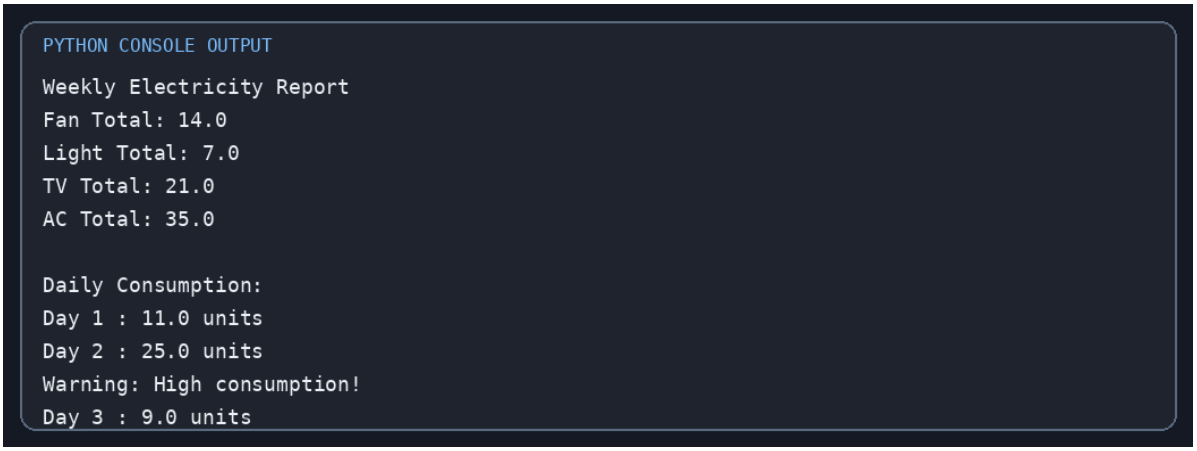
print("\nWeekly Electricity Report")
for i in range(4):
    total = 0
    for j in range(7):
        total += usage[i][j]
    print(appliances[i], "Total:", total)

print("\nDaily Consumption:")
for j in range(7):
    total = 0
    for i in range(4):
        total += usage[i][j]
    print("Day", j + 1, ":", total, "units")
    if total > 20:
        print("Warning: High consumption!")
```

8. Test Cases

Test	Input	Expected Result
1	All four appliances use 1 unit each on Day 1	Day 1 total is 4 units; no warning
2	Day 2 appliance values total 25	Day 2 total is 25 units; warning displayed

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program stores appliance usage in a matrix and calculates weekly and daily electricity consumption.

Project 49 – Tuple-Based Product Catalogue Audit

Concept: Tuples, lists, loops, conditions and unpacking

1. Problem Statement

Maintain a product catalogue using tuples containing product ID, price and stock quantity. Display product details, calculate stock value, and identify repeated product IDs or products with low stock.

2. Objectives

- Store fixed product records as tuples.
- Unpack tuple values inside a loop.
- Perform simple catalogue checks using conditions.

3. Assumptions

- Each product tuple contains ID, price and quantity in that order.
- A product is considered low-stock when quantity is below 6.
- The catalogue may contain repeated product IDs.

4. Algorithm

1. Create a list of product tuples.

2. Display each product's details.

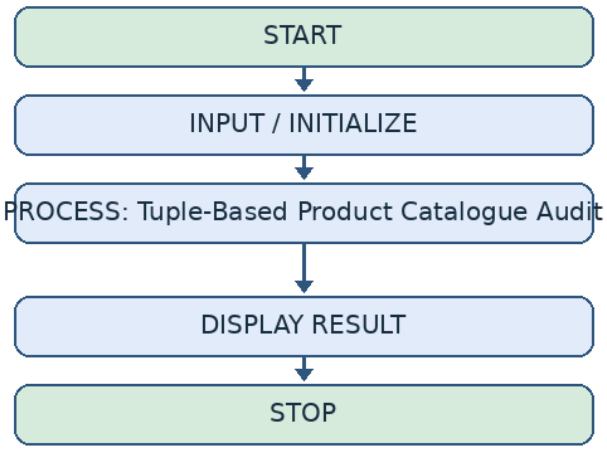
3. Calculate the stock value as price multiplied by quantity.

4. Check for repeated product IDs.

5. Display products whose quantity is below six.

6. Display the total catalogue stock value.

5. Flowchart



6. Pseudocode

```
START
CREATE list of product tuples (ID, price, quantity)
SET catalogue_total = 0
FOR each product tuple
    UNPACK ID, price, quantity
```

```
    DISPLAY product details
    ADD price * quantity to catalogue_total
    IF quantity < 6, DISPLAY low stock
COMPARE product IDs to find duplicates
DISPLAY catalogue_total
STOP
```

7. Python Code

```
products = [
    ("P101", 100, 10),
    ("P102", 250, 5),
    ("P103", 150, 4),
    ("P104", 200, 8),
    ("P101", 100, 10)
]

print("Product Catalogue Audit")
catalogue_total = 0

for product in products:
    product_id, price, quantity = product
    value = price * quantity
    catalogue_total += value
    print(product_id, "Price:", price,
          "Quantity:", quantity, "Stock Value:", value)

    if quantity < 6:
        print("Low stock:", product_id)

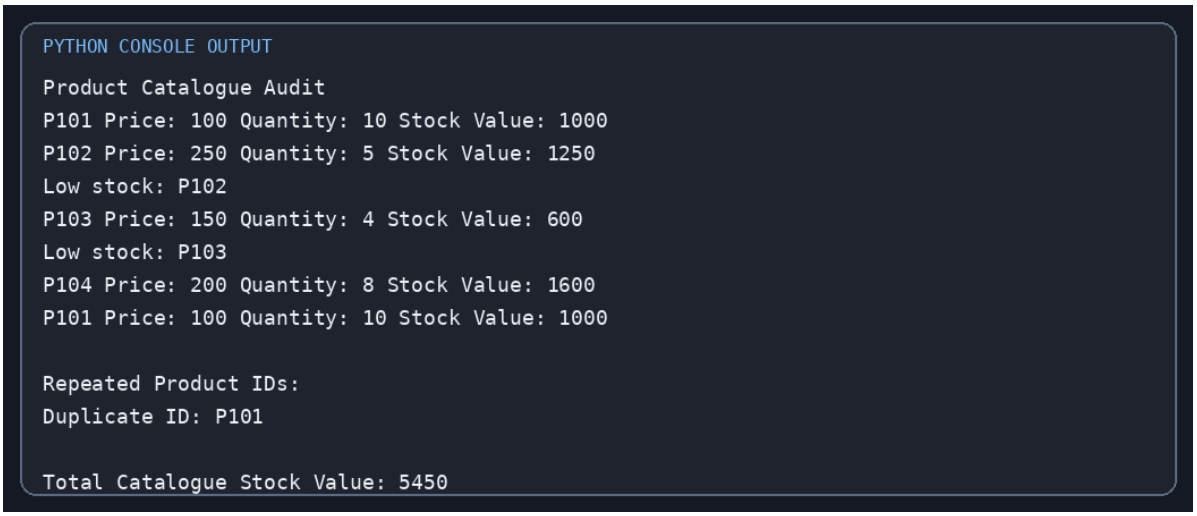
print("\nRepeated Product IDs:")
for i in range(len(products)):
    for j in range(i + 1, len(products)):
        if products[i][0] == products[j][0]:
            print("Duplicate ID:", products[i][0])

print("\nTotal Catalogue Stock Value:", catalogue_total)
```

8. Test Cases

Test	Input	Expected Result
1	P102 quantity 5	Low stock warning for P102
2	P101 appears twice	Duplicate ID P101
3	Price 100, quantity 10	Stock value 1000

9. Output Screenshot



Sample output for the test input shown above.

10. Conclusion

The program uses tuples to represent product records and audits stock levels, duplicate IDs and total stock value.